

DVC Hands-On Lab

Versioning the Wine Prediction Dataset with Git + DVC + AWS S3

MTech MLOps Training Notes — Windows 10/11 + VS Code Edition

This lab walks through the exact demo from the lecture — tracking a wine-quality CSV with DVC, pushing it to an AWS S3 bucket, and keeping Git and S3 in sync as the dataset changes — adapted for a Windows laptop using the VS Code integrated terminal. Topic 1 below adds a full beginner walkthrough for setting up AWS S3 and the AWS CLI from scratch.

MLOps Lifecycle Context: Data → Model Training → Evaluation → Deployment → Monitoring. This lab covers the Data stage end-to-end: versioning the dataset (DVC), storing it remotely (AWS S3), and keeping code + pointers in sync (Git).

Topic 1: Setting Up AWS S3 and the AWS CLI (Beginner, End-to-End)

What is this?

Before DVC can push your dataset anywhere, you need a place in the cloud to store it, and your laptop needs permission to talk to that place. This topic sets up exactly that: an AWS S3 bucket (the storage location) and the AWS CLI configured with credentials (the permission), both from a completely blank AWS account.

Why does it matter?

The transcript's DVC demo only works once 'DVC remote' points at a real S3 bucket and your machine is authorized to read/write to it. Skipping or misconfiguring this step is the single most common reason dvc push fails for beginners, so it's worth doing carefully, once, end-to-end.

How does it work?

Step 1 — Sign in to the AWS Console

1. Go to <https://aws.amazon.com> and sign in (or create a free-tier account if you don't have one; AWS asks for a credit/debit card even for the free tier, but S3 storage for a tiny CSV file costs a fraction of a cent per month).
2. Once signed in, you land on the AWS Management Console — the web dashboard for every AWS service.

Step 2 — Create an IAM user and access keys

Instead of using your main account's root credentials everywhere, AWS recommends creating a separate IAM (Identity and Access Management) user just for this kind of programmatic access.

1. In the Console search bar, type IAM and open the IAM service.
2. In the left sidebar, click Users, then click Create user.
3. Give it a name, e.g. dvc-mlops-user, and click Next.
4. On the permissions step, choose "Attach policies directly" and search for AmazonS3FullAccess (or a narrower S3 policy if your organization requires it), tick it, then click Next and Create user.
5. Click into the new user, open the Security credentials tab, scroll to Access keys, and click Create access key.
6. Choose Command Line Interface (CLI) as the use case, acknowledge the notice, click Next, then Create access key.
7. Download the .csv file (or copy both values right away): Access Key ID and Secret Access Key. The secret key is shown only once — if you lose it, you must generate a new key pair.

Step 3 — Create the S3 bucket

1. In the Console search bar, type S3 and open the S3 service.
2. Click Create bucket.

3. Give it a globally unique name — bucket names are shared across all AWS customers worldwide, so add your own name/initials, e.g. mlops-demo-<yourname>-dvc-bucket (the transcript uses mlops-demo-abhishek-dvc-bucket).
4. Pick an AWS Region close to you (e.g. ap-south-1 for Mumbai, us-east-1 for N. Virginia). Remember this region — you'll reuse it in aws configure.
5. Leave "Block all public access" turned ON (default) — your dataset should stay private; DVC talks to S3 using your credentials, not a public URL.
6. Leave the rest as default and click Create bucket. It appears in your bucket list within seconds.

Step 4 — Install the AWS CLI on Windows

1. Download the official 64-bit installer: <https://awscli.amazonaws.com/AWSCLIV2.msi>
2. Run the downloaded .msi and click through the installer with default options.
3. Open a new terminal (or a new VS Code integrated terminal — Terminal > New Terminal) so the updated PATH is picked up, and check the install: `aws --version`

Step 5 — Configure your credentials

1. In the same terminal, run: `aws configure`
2. Paste the Access Key ID when prompted.
3. Paste the Secret Access Key when prompted.
4. Enter the Default region name — use the same region you picked for the bucket (e.g. ap-south-1).
5. Enter the Default output format — json is a safe default.
6. These are saved locally to `C:\Users\<you>\.aws\credentials` and `C:\Users\<you>\.aws\config` — never commit this folder to Git.

Step 6 — Verify everything works

1. Run: `aws s3 ls`
2. You should see your new bucket listed with no errors. If you get an "Unable to locate credentials" or "Access Denied" error, re-run `aws configure` and double-check the IAM policy attached to your user.

Key Terms

Term	Plain-English Meaning
AWS Account	Your overall AWS billing/ownership identity — has full (root) access to everything.
IAM	Identity and Access Management — AWS's system for creating separate users/roles with limited permissions instead of always using the root account.
IAM User	A named identity (e.g. dvc-mlops-user) you create so you don't have to use root credentials day-to-day.

Term	Plain-English Meaning
Access Key ID / Secret Access Key	A pair of long strings that act like a username and password for programs (like DVC or the AWS CLI) instead of a human logging into a website.
S3 Bucket	A container in Amazon S3 (Simple Storage Service) where files are stored; bucket names must be globally unique.
Region	The physical AWS data-center location (e.g. ap-south-1 = Mumbai) where your bucket's data actually lives.
AWS CLI	A command-line program that lets you talk to AWS services (S3, IAM, etc.) from your terminal instead of the web console.
aws configure	The command that stores your Access Key ID, Secret Access Key, region, and output format locally so the CLI (and DVC) can authenticate automatically.

Simple Analogy

Think of AWS like a bank. Your AWS account is the bank itself. The root user is the bank's master key — powerful, but too risky to hand out for routine errands. An IAM user is like a personal locker ID the bank issues you. The Access Key ID and Secret Access Key are the key and PIN for that locker. An S3 bucket is the actual storage locker where you keep your belongings (your dataset). Running `aws configure` is like telling your own briefcase "remember this key and PIN" so it can open the locker for you automatically every time, without you standing at the counter.

Example

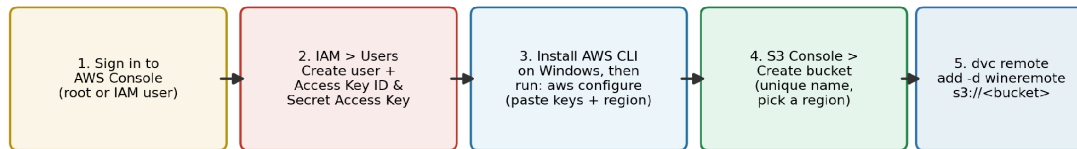
Commands you'll type in the VS Code terminal during this setup:

```
aws --version
# aws-cli/2.x.x Windows/10

aws configure
AWS Access Key ID [None]: AKIAxxxxxxxxxxxxxxxx
AWS Secret Access Key [None]: wJa1rxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Default region name [None]: ap-south-1
Default output format [None]: json

aws s3 ls
# 2026-07-01 10:15:32 mlops-demo-abhishek-dvc-bucket
```

Setting Up AWS S3 for DVC (Beginner Path)



Quick Recap

- Create an IAM user (not root) and generate an Access Key ID + Secret Access Key for CLI use.
- Create a globally-unique S3 bucket in a region you'll remember, with public access blocked.
- Install the AWS CLI on Windows via the official MSI installer.
- Run `aws configure` once to store your keys, region, and output format locally.
- Verify with `aws s3 ls` before touching DVC at all — this isolates AWS problems from DVC problems.

Common Exam/Interview Questions

Q1: Why use an IAM user instead of the AWS root account for DVC's S3 access?

A: The IAM user's permissions can be scoped (e.g., S3 only) and revoked independently, so a leaked key doesn't compromise the entire AWS account the way root credentials would.

Q2: Where does `aws configure` store your credentials, and should that folder ever go into Git?

A: In the `.aws` folder under your Windows user profile (`C:\Users\<you>\.aws`). It should never be committed to Git — credentials must be configured locally on each machine, not shared through source control.

Q3: What's the quickest way to confirm your AWS CLI + S3 setup is working before involving DVC?

A: Run `aws s3 ls` and confirm your bucket appears without an authentication error.

Topic 2: Hands-On Lab — Versioning the Wine Dataset (Git + DVC + S3)

What is this?

This is the practical, command-by-command walkthrough from the lecture: create a project, track a wine-quality CSV dataset with DVC, push it to the S3 bucket from Topic 1, and see exactly what changes in Git versus S3 every time the dataset is edited — all run from the VS Code terminal on Windows instead of the trainer's Mac/Linux terminal.

Why does it matter?

This is the same workflow used in real ML teams: the CSV changes constantly (cleaning, feature engineering, new records), and every teammate must be able to tell — with certainty — exactly which version of the dataset is "the one" currently being used by the project, without emailing files around.

How does it work?

1. Create the project and open it in VS Code

```
mkdir wine-prediction-demo
cd wine-prediction-demo
code .
# In VS Code: Terminal > New Terminal (opens PowerShell inside the folder)
```

2. Add the dataset

Create a data folder and place the wine_sample.csv file inside it (from the Udemy resources or the GitHub repo: github.com/iam-veeramalla/Wine-Prediction-Model). This is a simple dataset: input features (like acidity, sugar, alcohol, etc.) and one output column — wine quality, where a low number means a weak wine and a high number (e.g., 7) means a strong wine.

```
mkdir data
# copy wine_sample.csv into the data folder
```

3. Initialize Git

The dataset itself will NOT be tracked by Git — only code and DVC's small metadata files will be.

```
git init
```

4. Create and activate a virtual environment

Always use a virtual environment per project so its dependencies don't clash with other projects. On Windows, the activation command differs from the Mac/Linux "source .venv/bin/activate" shown in the lecture:

```
python -m venv venv

# PowerShell:
venv\Scripts\Activate.ps1

# If PowerShell blocks the script with an execution-policy error, run once:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
# Command Prompt (cmd.exe) instead of PowerShell:
venv\Scripts\activate.bat
```

5. Install and initialize DVC

```
python -m pip install dvc
dvc init
# Creates a .dvc/ folder (config, cache, .gitignore) and a .dvcignore file
# - this turns the folder into a DVC repo, just like git init does for Git
```

6. Track the dataset with DVC

```
dvc add data/wine_sample.csv
```

This creates `data/wine_sample.csv.dvc` — a small text file holding the dataset's metadata and MD5 checksum — and automatically adds `data/wine_sample.csv` to `data/.gitignore` so Git never tracks the raw CSV. Inspect it with:

```
Get-Content data\wine_sample.csv.dvc

# outs:
# - md5: 0618e7a101c8d57177eb6c6d964c674f
#   size: 320
#   hash: md5
#   path: wine_sample.csv
```

7. Add the S3 bucket as a DVC remote

With the bucket and AWS CLI from Topic 1 ready, connect DVC to it — this is DVC's equivalent of `git remote add`:

```
dvc remote add -d wineremote s3://mlops-demo-iftekar-dvc-bucket

# -d marks it as the default remote
# this writes to .dvc/config:
# [core]
#   remote = wineremote
# ['remote "wineremote"']
#   url = s3://mlops-demo-abhishek-dvc-bucket
```

8. Install S3 support for DVC

Running `dvc push` at this point fails with a missing-dependency error, because S3 support is a separate plugin:

```
python -m pip install dvc-s3
```

9. Push the dataset to S3

```
dvc push
# Uploads the actual CSV bytes to:
# s3://mlops-demo-abhishek-dvc-bucket/files/md5/06/18e7a101c8d57177eb6c6d964c674f
```

Check the S3 console: inside the bucket, under `files/md5/`, you'll see a folder named after the checksum, containing the uploaded CSV — downloadable for verification.

10. Commit the metadata (not the data) to Git

```
git add data/wine_sample.csv.dvc data/.gitignore .dvc/config .dvcignore
```

```
train.py utils.py requirements.txt
git commit -m "Track wine dataset v1 with DVC"
```

```
# optional – connect to the GitHub repo and push code + pointers:
git remote add origin https://github.com/<your-username>/Wine-Prediction-Model.git
git push -u origin main
```

Git now holds the .dvc file and .dvc/config (pointers to the data and the remote), plus your training code — never the CSV itself.

11. Make a change and create version 2

In the lecture, a new row is added to the CSV (an alcohol value changed from 0.86 to 0.87). Edit the file in VS Code, then repeat the add → commit → push cycle:

```
dvc add data/wine_sample.csv
# checksum in wine_sample.csv.dvc changes – this is now version 2 (11 rows)

git add data/wine_sample.csv.dvc
git commit -m "New version of dataset - added entry"
dvc push
```

12. Make another change and create version 3

The lecture then removes a duplicate-looking row to clean the data — same cycle again:

```
# edit data/wine_sample.csv in VS Code, remove the row
dvc add data/wine_sample.csv
git add data/wine_sample.csv.dvc
git commit -m "New version of dataset - removed duplicate"
dvc push
```

Back in the S3 console, the files/md5/ folder now shows multiple checksum entries — one per version ever pushed. Git's committed .dvc file always points at the current one; older S3 entries simply stay available as history.

13. How a teammate gets the exact right version

```
git clone https://github.com/<your-username>/Wine-Prediction-Model.git
cd Wine-Prediction-Model
python -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt dvc dvc-s3
aws configure # their own IAM access key / secret key
dvc pull
# reads data/wine_sample.csv.dvc from the cloned repo
# and downloads exactly that checksum's file from S3
```

This is the core reason the .dvc file must live in Git: with 100 versions sitting in the S3 bucket, Git is the single source of truth for which one the team is actually using right now.

Key Terms

Term	Plain-English Meaning
dvc init	Turns the current folder into a DVC repository (creates the <code>.dvc/</code> folder), the same way <code>git init</code> does for Git.
dvc add <file>	Starts tracking a file with DVC; creates a <code><file>.dvc</code> pointer file and adds the real file to <code>.gitignore</code> automatically.
.dvc file	A small YAML file (e.g. <code>wine_sample.csv.dvc</code>) holding the tracked file's path, size, and MD5 checksum for that version.
MD5 checksum	A unique fingerprint calculated from a file's contents; any change to the file produces a different checksum, which is how DVC tells versions apart.
dvc remote add -d	Registers a remote storage location (here, an S3 bucket) as DVC's default push/pull target — the DVC equivalent of <code>git remote add</code> .
.dvc/config	The file that records which remote is default and its URL (<code>s3://bucket-name</code>); this gets committed to Git so everyone knows where the data lives.
dvc push / dvc pull	Upload the currently tracked file version to the remote / download the version referenced by the local <code>.dvc</code> file from the remote.
dvc-s3	The DVC plugin that adds S3 support; without it, <code>dvc push/pull</code> against an <code>s3://</code> remote fails with a missing-dependency error.

Simple Analogy

Every time you repack a suitcase (edit the CSV) and check it into the storage depot (S3), the depot prints a new tracking number (the MD5 checksum) for that exact suitcase. You never carry the suitcase itself — you only keep the small tracking receipt (the `.dvc` file) in your notebook (Git). Even if the depot is holding a hundred old suitcases from a hundred old trips, your notebook's receipt tells you — and anyone who borrows your notebook — precisely which suitcase is the current one to pick up.

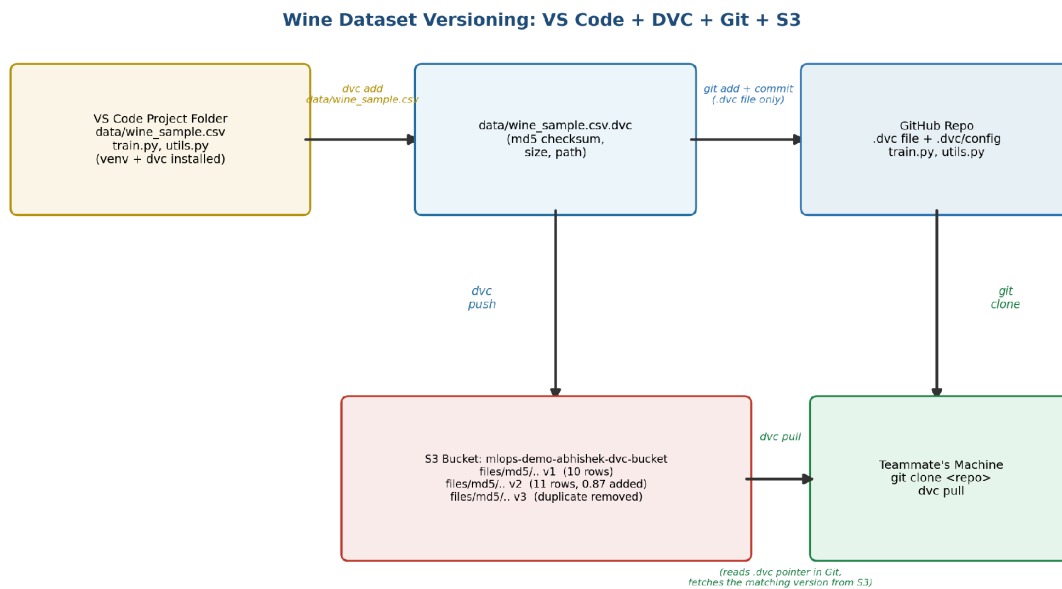
Example — the exact files this lab produces

data/wine_sample.csv.dvc (created by `dvc add`):

```
outs:
- md5: 0618e7a101c8d57177eb6c6d964c674f
  size: 320
  hash: md5
  path: wine_sample.csv
```

.dvc/config (created by `dvc remote add -d`):

```
[core]
  remote = wineremote
['remote "wineremote"']
  url = s3://mlops-demo-abhishek-dvc-bucket
```

.gitignore (auto-updated so Git never tracks the raw CSV):`data/wine_sample.csv`

Golden rule: Git = source of truth for WHICH version to use. S3 = storage for the ACTUAL file bytes.

Quick Recap

- git init tracks code; dvc init + dvc add tracks the dataset — the two never overlap on the same file.
- dvc add creates a small .dvc pointer file (path, size, MD5 checksum) and auto-adds the real CSV to .gitignore.
- dvc remote add -d s3://bucket connects DVC to your S3 bucket; the dvc-s3 plugin must be installed separately.
- dvc push uploads the actual CSV bytes to S3; git add/commit stores only the .dvc file and .dvc/config.
- Every edit to the CSV → new dvc add → new checksum → new dvc push → new commit: S3 accumulates every version, Git's latest commit says which one is current.
- Teammates run git clone then dvc pull to fetch exactly the version the team is using — never a random one from S3.

Common Exam/Interview Questions

Q1: After editing a CSV that's already tracked by DVC, what two commands must you run, and in what order?

A: dvc add <file> first (to update the checksum in the .dvc file), then dvc push (to upload the new version to the remote) — plus git add/commit on the updated .dvc file so Git records the new pointer.

Q2: If the S3 bucket ends up with 100 versions of a dataset, how does a data scientist know which one to use?

A: By checking the .dvc file currently committed in Git — its checksum is the source of truth for which S3 version is "active", regardless of how many other versions exist in the bucket.

Q3: Why does dvc push fail immediately after setting up a fresh S3 remote, and how do you fix it?

A: DVC needs the separate dvc-s3 package to talk to S3; installing it with `pip install dvc-s3` resolves the error.

Q4: On Windows, what replaces the "source .venv/bin/activate" command from the lecture?

A: `venv\Scripts\Activate.ps1` in PowerShell (or `venv\Scripts\activate.bat` in Command Prompt).